

Relatório do Projeto 2: RSA

Ana Paula Gomes de Matos (160023629), André Milhomem Araújo de Godoi (202033427),
Geilson dos Santos Sa (231006239)

Resumo—Este relatório apresenta projeto de desenvolvimento de programa que utiliza o sistema de criptografia Rivest–Shamir–Adleman (RSA) combinado com os esquemas Optimal Asymmetric Encryption Padding (OAEP) e Probabilistic Signature Scheme (PSS) para cifrar e decifrar mensagens e para gerar e verificar assinaturas digitais.

Palavras-chave—Rivest–Shamir–Adleman (RSA), Optimal Asymmetric Encryption Padding (OAEP), Probabilistic Signature Scheme (PSS).

I. INTRODUÇÃO

O sistema RSA, descrito por Ron Rivest, Adi Shamir e Leonard Adleman em 1977, consiste em um esquema de criptografia assimétrico que se baseia em um par de chaves (uma pública; outra privada) e combina propriedades de aritmética modular e aritmética de números primos para garantir confidencialidade, integridade e autenticidade de mensagens [1].

Na atualidade, como camada adicional de segurança, o sistema RSA é complementado pelos esquemas Optimal Asymmetric Encryption Padding (OAEP), para cifragem e decifragem de mensagens, e Probabilistic Signature Scheme (PSS), para geração e verificação de assinatura digital [1].

Neste projeto, desenvolvemos um programa que implementa, com adaptações, os algoritmos RSAES-OAEP e RSASSA-PSS, descritos na RFC 8017 [2], para cifrar e decifrar mensagens e para gerar e verificar assinaturas digitais, respectivamente. A fundamentação teórica, o ambiente experimental e os resultados obtidos são descritos a seguir.

II. FUNDAMENTAÇÃO TEÓRICA

A. RSA

O sistema RSA se baseia na utilização de uma chave pública e uma chave privada, obtidas conforme a especificação abaixo.

Seja $\phi(x)$, com $x \in \mathbb{Z}^+$, o número de inteiros positivos menores que x e coprimos de x (função totiente de Euler). Podemos provar que $\phi(x) = x \prod_{i=1}^k (1 - \frac{1}{y_i})$, sendo $y_1, \dots, y_k$ os k distintos números primos que dividem x [3].

Sejam p e q números primos quaisquer distintos e $n = p \cdot q$. Temos que $\phi(n) = n[(1 - \frac{1}{p})(1 - \frac{1}{q})] = n[(\frac{p-1}{p})(\frac{q-1}{q})] = n[\frac{(p-1)(q-1)}{pq}] = n[\frac{(p-1)(q-1)}{n}] = (p-1)(q-1)$.

Seja e um número inteiro positivo coprimo de $\phi(n)$, isto é, $\text{mdc}(e, \phi(n)) = 1$. Pelo Teorema de Bézout, sabemos que e e $\phi(n)$ são coprimos se, e somente se, e é inversível módulo $\phi(n)$, isto é, que $\text{mdc}(e, \phi(n)) = 1 \iff \exists e^{-1} \in \mathbb{Z}^+ : ee^{-1} \equiv 1 \pmod{\phi(n)}$. Nesse contexto, seja $d \in \mathbb{Z}^+$ o inverso de e módulo $\phi(n)$, isto é, $ed \equiv 1 \pmod{\phi(n)}$ ou ainda $ed = k\phi(n) + 1, k \in \mathbb{Z}^+$.

Sejam m um número inteiro positivo que represente uma mensagem e c um número inteiro positivo que represente a cifra dessa mensagem m .

No sistema RSA, a operação de cifragem é assim definida: $c = m^e \pmod{n}$. Já a operação de decifragem é realizada da seguinte maneira: $m = c^d \pmod{n}$.

Para que o sistema RSA funcione, precisamos garantir a inversibilidade das operações, isto é, provar que, ao decifrar um texto anteriormente cifrado, podemos obter o texto original.

Pelo Teorema de Euler, sabemos que $\text{mdc}(m, n) = 1 \implies m^{\phi(n)} \equiv 1 \pmod{n}$. Além disso, sabemos que $m \pmod{n} = m, \forall m < n$.

Supondo que m e n são coprimos e que $m < n$, observamos o seguinte:

$$\begin{aligned}
 c^d \pmod{n} &= \\
 (m^e \pmod{n})^d \pmod{n} &= \\
 m^{ed} \pmod{n} &= \\
 m^{k\phi(n)+1} \pmod{n} &= \\
 m^{k\phi(n)} m \pmod{n} &= \\
 (m^{k\phi(n)} \pmod{n})(m \pmod{n}) \pmod{n} &= \\
 ((m^{\phi(n)})^k \pmod{n})(m \pmod{n}) \pmod{n} &= \\
 ((m^{\phi(n)} \pmod{n})^k \pmod{n})(m \pmod{n}) \pmod{n} &= \\
 (1^k \pmod{n})(m \pmod{n}) \pmod{n} &= \\
 (1 \pmod{n})(m \pmod{n}) \pmod{n} &= \\
 1(m \pmod{n}) \pmod{n} &= \\
 (m \pmod{n}) \pmod{n} &= \\
 m \pmod{n} &= \\
 m
 \end{aligned}$$

Podemos, aliás, provar que esse resultado ($c^d \pmod{n} = m$) vale mesmo na remota hipótese em que m e n não sejam

coprimos¹.

Notamos que a suposição de que $m < n$ é primordial para garantir a injetividade da operação de cifragem, isto é, para assegurar que para cada mensagem m existe uma única mensagem cifrada c correspondente.

Adicionalmente, observamos que, como $ed = de, \forall e, d \in \mathbb{Z}^+$, se utilizarmos o expoente d na operação inicial, podemos utilizar o expoente e para invertê-la. Com base nessa propriedade, podemos definir a operação de assinatura como $c = a^d \bmod n$ e a operação de verificação de assinatura como $a = c^e \bmod n$, sendo a o valor de uma função de *hash* aplicada sobre a mensagem m .

No sistema RSA, o par (n, e) é denominado chave pública e o par (n, d) , chave privada. Nas operações de cifragem e decifragem de uma mensagem, utilizam-se, nessa ordem, as chaves pública e privada do destinatário. Já nas operações de assinatura e verificação de assinatura, utilizam-se, nessa ordem, as chaves privada e pública do remetente.

Em razão da injetividade da operação de cifragem, o sistema RSA assume caráter determinístico: dada uma mensagem m e dadas as chaves pública (n, e) e privada (n, d) , o resultado c da operação de cifragem será sempre o mesmo.

Essa característica representa uma vulnerabilidade do sistema RSA e ensejou a criação de esquemas de *padding*, como o OAEP e o PSS (que veremos a seguir), com o objetivo de adicionar uma camada de aleatoriedade ao sistema.

B. OAEP

O esquema OAEP consiste em uma série de transformações que atuam sobre a mensagem m previamente à operação de cifragem pelo sistema RSA. A Figura 1, extraída de [2], sintetiza esse processo.

O processo consiste dos seguintes passos:

- 1) Sobre uma *label* pré-determinada, aplica-se uma função de *hash*, obtendo-se a sequência *lHash* com $hLen$ octetos (*bytes*).
- 2) Dado o componente n (com k octetos) das chaves do destinatário, limitamos o tamanho $mLen$ da mensagem m a ser codificada a, no máximo, $k - 2hLen - 2$ octetos.

¹Mesmo que m e n não sejam coprimos, podemos provar que $c^d \bmod n = m$ ou, mais especificamente, que $m^{ed} \bmod n = m$. Como $n = pq$ e p e q são primos, a única hipótese em que m e n não são primos é a situação em que m é múltiplo de p ou de q . Suponhamos que m seja múltiplo de p ($m = kp, k \in \mathbb{Z}^+$). Como $m < n = pq$, nesse cenário, m é necessariamente coprimo de q . Bifurquemos a prova em dois ramos: a) $m^{ed} \equiv m \pmod{p}$; e b) $m^{ed} \equiv m \pmod{q}$. O item a) pode ser provado assim: $m^{ed} \bmod p = (m \bmod p)^{ed} \bmod p = (kp \bmod p)^{ed} \bmod p = 0^{ed} \bmod p = 0 \bmod p = 0 = kp \bmod p = m \bmod p$. Quanto ao item b), sabemos, pelo Teorema de Euler, que $\text{mdc}(m, q) = 1 \implies m^{\phi(q)} \equiv 1 \pmod{q}$ e, como q é primo, que $\phi(q) = q - 1$. Assim, $m^{\phi(q)} = m^{q-1} \equiv 1 \pmod{q}$. Sabemos ainda que $\phi(n) = (p-1)(q-1)$. Com base nessas premissas, observamos o seguinte: $m^{ed} \bmod q = m^{j\phi(n)+1} \bmod q = m^{j(p-1)(q-1)+1} \bmod q = (m^{j(p-1)(q-1)} \bmod q)(m \bmod q) \bmod q = ((m^{q-1})^j \bmod q)(m \bmod q) \bmod q = ((m^{q-1}) \bmod q)^j \bmod q(m \bmod q) \bmod q = ((1 \bmod q)^{j(p-1)} \bmod q)(m \bmod q) \bmod q = (1^{j(p-1)} \bmod q)(m \bmod q) \bmod q = (1 \bmod q)(m \bmod q) \bmod q = 1(m \bmod q) \bmod q = (m \bmod q) \bmod q = m \bmod q$. Pelo Teorema Chinês do Resto, como $m^{ed} \equiv m \pmod{p}$ e $m^{ed} \equiv m \pmod{q}$, podemos afirmar que $m^{ed} \equiv m \pmod{pq}$.

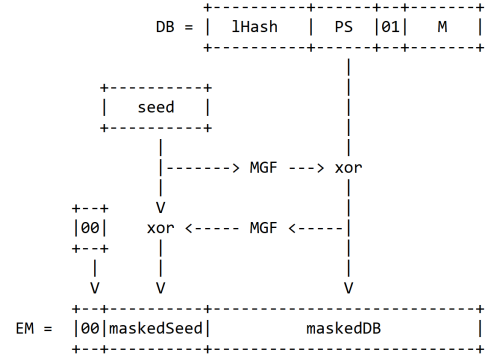


Figura 1. Esquema OAEP

- 3) Fixado o tamanho da mensagem, calculamos o *padding* PS , que consiste de $k - mLen - 2hLen - 2$ octetos de zeros ($0x00$).
- 4) Geramos o bloco de dados DB a partir da concatenação $lHash||PS||0x01||m$, sendo $0x01$ um octeto de separação entre o *padding* de zeros e a mensagem m . O bloco de dados DB terá tamanho fixo igual a $k - hLen - 1$.
- 5) Aleatoriamente, geramos a semente $seed$, do mesmo tamanho de $lHash$, isto é, $hLen$.
- 6) Sobre a semente $seed$, aplicamos uma função geradora de máscara (MGF), que produzirá uma máscara $dbMask$ do tamanho de DB , isto é, $k - hLen - 1$.
- 7) Realizamos uma operação XOR entre DB e $dbMask$ para obter o bloco de dados mascarado $maskedDB$.
- 8) Sobre o bloco de dados mascarado $maskedDB$, aplicamos a mesma função geradora de máscara anteriormente descrita (MGF), agora para produzir uma máscara $seedMask$ de tamanho $hLen$.
- 9) Realizamos uma operação XOR entre $seed$ e $seedMask$ para obter a semente mascarada $maskedSeed$.
- 10) Por fim, realizamos a seguinte concatenação: $0x00||maskedSeed||maskedDB$. O resultado dessa operação é a mensagem codificada EM . Sobre ela aplicaremos a operação de cifragem que caracteriza o sistema RSA.

Após a decifragem de uma mensagem pelo sistema RSA, para obter a mensagem original devemos reverter o processo de codificação acima descrito da seguinte maneira:

- 1) Decompomos a mensagem codificada em $0x00$ (primeiro octeto), $maskedSeed$ ($hLen$ octetos seguintes) e $maskedDB$ (demais octetos).
- 2) Aplicamos a função geradora de máscara (MGF) sobre $maskedDB$ para obter $seedMask$.
- 3) Realizamos a operação XOR entre $maskedSeed$ e $seedMask$ para obter $seed$.
- 4) Aplicamos a função geradora de máscara (MGF) sobre $seed$ para obter $dbMask$.
- 5) Realizamos a operação XOR entre $maskedDB$ e $dbMask$ para obter DB .
- 6) Decompomos DB em $lHash$ (primeiros $hLen$ octetos), PS (octetos de zero seguintes), $0x01$ (octeto de separação) e m (restante).

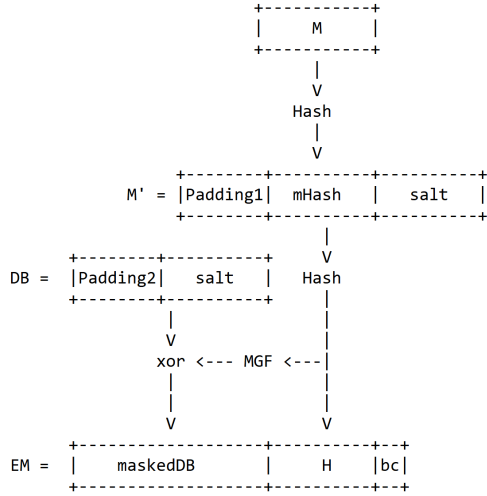


Figura 2. Esquema PSS

ção e, por fim, m .

Ao longo do processo de decodificação de EM , realizamos diversas checagens para garantir a integridade da mensagem: a) checamos se o primeiro octeto da mensagem codificada EM é igual a 0x00; b) verificamos se $lHash$ corresponde à aplicação da função de hash à *label*; e c) conferimos a existência do octeto de separação 0x01 em DB .

C. PSS

Similarmente ao processo de cifragem e decifragem, aplicamos ao processo de geração e verificação de assinatura digital um esquema (o PSS) para acrescentar um elemento de aleatoriedade, previamente à operação de cifragem pelo sistema RSA. A Figura 2, extraída de [2], sintetiza a lógica do esquema PSS.

A aplicação do esquema PSS se desdobra nos seguintes passos:

- 1) Sobre a mensagem m que desejamos assinar, aplicamos uma função de hash para obter a sequência $mHash$ com $hLen$ octetos (bytes).
- 2) Aleatoriamente, geramos a sequência $salt$ de tamanho de $sLen$.
- 3) Produzimos a sequência $Padding1$ com oito octetos 0x00.
- 4) Realizamos a concatenação $Padding1||mHash||salt$ e a denominamos m' .
- 5) Sobre m' aplicamos uma função de hash para obter a sequência H , com o mesmo tamanho de $mHash$, isto é, $hLen$.
- 6) Estipulamos o tamanho útil $emBits$ (em bits) da assinatura codificada EM , observando que ele deve ser menor que o tamanho $modBits$ (em bits) do componente n das chaves, isto é, garantindo que $emBits < modBits$.
- 7) Calculamos a quantidade $emLen$ de octetos necessários para representar $emBits$ ($emLen = \lceil emBits/8 \rceil$).
- 8) Geramos a sequência PS com $emLen - sLen - hLen - 2$ octetos 0x00 e a concatenamos com o octeto separador 0x01, ao final, formando $Padding2$.

- 9) Compomos o bloco de dados DB por meio da concatenação $Padding2||salt$.
- 10) Aplicamos uma função geradora de máscara (MGF) sobre H para gerar uma máscara $dbMask$ do mesmo tamanho de DB , isto é, $emLen - hLen - 1$ octetos.
- 11) Para gerar a sequência $maskedDB$, realizamos uma operação XOR entre DB e $dbMask$ e, em seguida, zeramos os $8 \cdot emLen - emBits$ primeiros bits do resultado.
- 12) Produzimos a concatenação $maskedDB||H||0xbc$. Assim, obtemos a assinatura codificada EM , que será cifrada pelo sistema RSA.

Dadas uma mensagem m , em claro, e uma assinatura EM gerada a partir de m pelo esquema PSS, podemos verificar a integridade e a autenticidade da mensagem assinada da seguinte maneira:

- 1) Decompomos a assinatura codificada EM em três partes: 0xbc (o último octeto); H (os $hLen$ octetos antecedentes a 0xbc); e $maskedDB$ (os octetos iniciais restantes).
- 2) Aplicamos a função geradora de máscaras (MGF) sobre H para obter $dbMask$.
- 3) Realizamos uma operação XOR entre $maskedDB$ e $dbMask$ para obter DB .
- 4) Decompomos DB para obter $salt$ (os $sLen$ últimos octetos).
- 5) Aplicamos a função de hash sobre m para obter $mHash$.
- 6) Geramos o $Padding1$ a partir da concatenação de oito octetos 0x00.
- 7) Realizamos a concatenação $Padding1||mHash||salt$ para obter m' .
- 8) Aplicamos a função de hash sobre m' e comparamos o resultado a H : se o resultado for igual, estão verificadas a integridade e a autenticidade da mensagem assinada.

Durante o processo de decodificação de EM , também realizamos uma série de checagens: a) observamos se o octeto final é igual a 0xbc; b) checamos se os $8 \cdot emLen - emBits$ primeiros bits de EM estão zerados; e c) verificamos se $Padding2$ é composto por octetos 0x00 no início e pelo octeto separador 0x01 ao final.

III. AMBIENTE EXPERIMENTAL E ANÁLISE DE RESULTADOS

A. Descrição do cenário

Para ilustrar o funcionamento do sistema RSA, complementado pelos esquemas OAEP e PSS, implementamos um programa em linguagem Python (versão 3.13.13) com as funcionalidades de geração de chaves, cifragem e decifragem de mensagens e geração e verificação de assinaturas.

No código, a funcionalidade de geração de chaves é implementada pela função principal *gera_chaves* e pelas funções auxiliares *gera_primo*, *gera_e* e *aplica_miller_rebin*.

A função *gera_primo* gera aleatoriamente valores ímpares no intervalo $[2^{1024}, 2^{1025})$ como candidatos a p e q , e

invoca a função *aplica_miller_rebin* para testar, probabilisticamente, a primalidade desses valores, com base no algoritmo de Miller-Rebin.

A função *gera_e* parte de um número primo inicial (*E_CANDIDATO*, parâmetro que fixamos em 65537) e testa sua coprimidade com $\phi(n)$; em caso negativo, são considerados os números ímpares subsequentes até se encontrar um coprimo de $\phi(n)$.

As funcionalidades de cifragem e decifragem do sistema RSA são implementadas pela função *cifra_decifra* que utiliza a função *pow*, nativa da linguagem de programação Python, para realizar as operações de exponenciação modular.

O esquema OAEP é implementado pelas funções *empacota_oaep*, que realiza o processo de codificação, e *desempacota_oaep*, responsável pelo processo de decodificação.

Similarmente, o esquema PSS é implementado pelas funções *empacota_pss* e *desempacota_pss*. Nesse contexto, a função *assina_mensagem* é responsável por disparar o processo de codificação realizado por *empacota_pss*.

A função de *hash* utilizada nos diversos momentos do processo de codificação e decodificação dos esquemas OAEP e PSS é a função *sha3_256*, que gera *hashes* de 32 bytes e foi implementada pela biblioteca *hashlib*.

Como função geradora de máscaras (*MGF*), implementamos a função *aplica_MGF1*, que concatena *hashes* gerados pela função *sha3_256* com contadores para produzir máscaras nos tamanhos desejados.

Por meio do terminal, o usuário escolhe a opção de ação que deseja executar (1 - Gerar chaves públicas e privadas para A e B, 2 - Cifrar mensagem de A para B, 3 - Decifrar mensagem de A para B, 4 - Assinar mensagem de A para B ou 5 - Verificar assinatura de mensagem de A para B).

Os resultados são sempre impressos no terminal, meio pelo qual também se recebem as chaves (pública ou privada) e as mensagens (a cifrar, a decifrar ou assinada), sempre codificadas em base64.

As chaves pública (n, e) e privada (n, d), assim como a mensagem assinada, são informadas e recebidas em formato JSON, funcionalidade implementada pelas funções *formata_chave*, *desformata_chave* e *assina_mensagem*.

B. Análise de resultados

Para ilustrar o funcionamento do programa, selecionamos, inicialmente, a opção 1 - Gerar chaves públicas e privadas para A e B. Após execução do programa, obtivemos a seguinte resposta, com as chaves geradas:

```
===== CHAVE PÚBLICA DE A =====
eyJJogIjdQSczFhdHg0QjIwRWVlc2RMem1UQ
Qno2UE1NVnNjaDJGUHhPaWZBUUNjNGR1V0I0
ajBldkM5NWlxZGRlyUVldGRycGdLTnhKTGtw
ay9hd0xRQTNMcFRKU1p5SWdpSjRUM1g0R0JO
YzRYOHNhTW1LTkxlbXVGY2wxT3RtTCtmaXBv
cjVBejRNeG5rVTNjYldEcDFzbEZxQWorNk5z
Q2dpUVU2aHZ3MXduRXJqdTVnYVBOcG5NcEtI
VFBBSXh3U2FyQzB3cG9RNDJlYXBscE5WbHRp
```

```
NVRiY3Q3WUwxa2RqUTNPWm5PTVZUa2htR1dI
T08rMTh3bTA1MVNQOVBOU21NRGINTE9Ddy9y
MW5lYzF5R3BkNFZjMXNrK0lVT2huL1F4cmIw
Y2xCskpmZXM2WmlWVWJjall1dFlNaUEvcWtM
MzVDWUNZQ2xIMnZpcm16WERQeUhuWG5pUT09
liwgImUiOiAiQVFBQjI9
```

```
===== CHAVE PRIVADA DE A =====
eyJJogIjdQSczFhdHg0QjIwRWVlc2RMem1UQ
no2UE1NVnNjaDJGUHhPaWZBUUNjNGR1V0I0aj
BldkM5NWlxZGRlyUVldGRycGdLTnhKTGtway9
hd0xRQTNMcFRKU1p5SWdpSjRUM1g0R0JOYzRY
OHNhTW1LTkxlbXVGY2wxT3RtTCtmaXBvcjVBe
jRNeG5rVTNjYldEcDFzbEZxQWorNk5zQ2dpUV
U2aHZ3MXduRXJqdTVnYVBOcG5NcEtIVFBBSXh
3U2FyQzB3cG9RNDJlYXBscE5WbHRpNVRiY3Q3
WUwxa2RqUTNPWm5PTVZUa2htR1dIT08rMTh3b
TA1MVNQOVBOU21NRGINTE9Ddy9yMW5lYzF5R3
BkNFZjMXNrK0lVT2huL1F4cmIwY2xCskpmZXM
2WmlWVWJjall1dFlNaUEvcWtMMzVDWUNZQ2xI
MnZpcm16WERQeUhuWG5pUT09liwgImUiOiAiAe
UpaTzdqL2xmUUJScDhSWjJlXm5eDBHcXdyMl
cwU1NZR3VHeWVXN0RZSkE3L080dFNvVWxovSDF
tL2pGWtk2TDJCbnR5K0U1Q01taTNqeHJUQUFq
Y0NqOFpyNlg2MHlZL2JUc1pWZ3EvZU1lY3RBS
jc1Nm1RY0tydDhybUZuaGdWditFYTNHZZTIyde
doM0hiZzNDdnUyOGLISTRLK1pFQ0E0M1R2TGN
RUVUyOUFbnK21QdlBwUUhBQ3FMTnptZm5CSTRw
MzJlK1dHZ0E5SmVqWnNpeGYvdm5WUetKL3diW
mZiToS2WDNML0xVem1XSzZINGFZYUNDK0tCVD
QxYWZNC1JlcWtJQkloM2RVWIM3TDNldFpIdzF
lK1VrcVhlaXNleTFsZUFVSEtuaUVNVVNSSzBu
NGQ4MG54M1JTZGcrQUUNa3VOTHFreFVSeWJtN
UFuOVIvS1IRPT0ifQ==
```

```
===== CHAVE PÚBLICA DE B =====
eyJJogImZlT1NaMkpnTVpVa2RDT3ZNYmxlR
Hk1Z3phN1pPblpCVmVmYjMrUmw5dWR0ZUR3ak
JkVnBrNzNjZjBzZjZjZjZjZjZjZjZjZjZjZj
ObGM2VkUvRi9UeWFyRzNmT21Y2JXb0p6czBZ
cjRMcFBGL25YV0FjZjZjZjZjZjZjZjZjZjZjZj
HFBblZKUVhleTjZdiszSzf6YmIwdGswVFeraE
VYL3p6NVJmbHBmc0ErcWZ5YWR6U1djWlc2ZHY
zNWN4NUFvU1c3WkdWODJZMXRtUnRGVVGZ0QUxY
TS84SnZ6djBQYjd0bWk0Rk5KWnhHNXNEdmJtS
3VsMDRGZ3pUQXdjYlIHb2dV2dEcNbsb0RVZm
5wZXVUWnJtTy8ybmxYa1V5T29YNkd6eU5hRUN
yVknRSU5oNVM1SWliOXU1VVRReXpVdWkwTng4
ZUlwZjc4Q3hDamE3YXVPUT09liwgImUiOiAiQ
VFBQjI9
```

```
===== CHAVE PRIVADA DE B =====
eyJJogImZlT1NaMkpnTVpVa2RDT3ZNYmxlR
Hk1Z3phN1pPblpCVmVmYjMrUmw5dWR0ZUR3ak
JkVnBrNzNjZjBzZjZjZjZjZjZjZjZjZjZjZj
ObGM2VkUvRi9UeWFyRzNmT21Y2JXb0p6czBZ
cjRMcFBGL25YV0FjZjZjZjZjZjZjZjZjZjZjZj
HFBblZKUVhleTjZdiszSzf6YmIwdGswVFeraE
```

VYL3p6NVJmbHBmc0ErcWZ5YWR6U1djWlc2ZHY
zNWN4NUFvU1c3WkdWODJZMXRtUnRGVGZ0QUxY
TS84SnZ6djBQYjd0bWk0Rk5KWnhHNXNEdmJtS
3VsMDRGZ3pUQXjdYIiHbjdV2dEcnBsb0RVZm
5wZXVUWnJtTy8ybmXyA1V5T29YNkd6eU5hRUN
yVknRSU5oNVM1SWliOXU1VVRReXpVdWkwTng4
ZUlwZjc4Q3hDamE3YXVPUT09IiwgImQiOiAiT
UtPQmIGSTVDazh1WTNUNkZvTWbOVXBjK25zbz
BPNnB0YkE4Z3BlElVaNys2NnFJZVpPSUZCNEt
aZVBZTVp5WkF1TytFRFJzdFoxRzdlVHIwM0Z5
OUxHNnJRNvNaamNRVm5IVzBrRHazYjM5c1diN
GRGcExTbFVCWVdDNFpStZrkYzQwWWN4U3Z0Z3
l5WE5qZVlwTGRlelBSYIFuMGwvbU14UC9nNjh
rK21paVRJMDNvTTlVSGpaL2ZGSzN2MW9iSUNU
YWlkSDdyWfo3Unp2QTI0b3kvQXZtL3I2TnFET
3J6SUhGYXFHaFJ5clckRGU3TXA1aV3Y1JSY2
lTY204T2RPU2JuZEI4TFFCS3hvTnp0Q2RaMXl
FZk5UN3JNV3hsbU8xNG5ReXNtZldaSjdTR0h0
VDFEZEFOczRmZWtWaGdpSnorN1crSkx3RlcwU
k1seVkzaWxRPT0ifQ==

Geradas as chaves, acionamos a opção 2 - *Cifrar mensagem de A para B* e informamos a mensagem *Esta é uma mensagem de teste!* e a chave pública de B (destinatário). Como resposta, a mensagem cifrada recebida foi a seguinte:

Ys3aK50XGg+dg9+ImKYaNjXCQXxziESPTxOo
HvQyULI147r42q7zqqbnpOupcIceqpalCOfo
T4hminQCC0SRrSu4u3j+Ci640ThBv0W6CpWi
7h6RzbsmHGSrXXJxZWNEutd4DhEA3dKPudHb
A2DUyZEPmVa546d4n44WjhMGJOdqsJFIL6Ke
y110H72nZH+Invacna2mCQRVyTb/gG0fRHHG
mdVcsKxeondPM4FAFoh3Da2ts/MYoVp00aIl
RynCKtEbbcTcnM8IUKN6R3IetbeAlxJKptNU
YZseIfwPEgfCgoDczys5fsQVjAUBb20B406S
XkhOzenjdt6VDbMMHA==

Em seguida, selecionamos a opção 3 - *Decifrar mensagem de A para B* e informamos a mensagem cifrada e a chave privada de B (destinatário). Como resultado, recebemos o texto: *Esta é uma mensagem de teste!*

Em sequência, escolhemos a opção 4 - *Assinar mensagem de A para B* e informamos a mensagem *Esta é uma mensagem de teste!* e a chave privada de A (remetente). O resultado obtido foi a seguinte mensagem assinada:

eyJtZW5zYWdlbSI6ICJFc3RhIFx1MDBlOSB1b
WEgbWVuc2FnZW0gZGUgdGVzdGUhliwglmFzc2
luYXR1cmEiOiAiZGk0U3V3V3QxK3FHc0o2dT
HcWlUZUZMR1BubkRWdkJ4d115bHlHQVZvVG95
SW5hTGfWdndHBoWkoxay96YU4rc2R2Yllyb
XlNR3ZQZnhxTE1vamE5SkhHcTZpU2VKTFVNcE
toczZQbVBibi84S3BGNldSZU5Ub0o2c2dkeEp
QRnFpTW5VOXp6S1h2TjFtREIRZTFDSDhaSjBw
TIRIdm8reTJKaklaVWl3bUZTRDNEL1Fvdnhib
k1rWDd0c2w0ZUtdlpwbTdKRHgxZThkWEttS1
l6bEMxaHorK1BNSGxyl3FOclJRd1hTY3IKUS8
yVGxRdlRqNEdqZjdMZ24zdZB0SWdreGUyS2VN

eEp0dHBJcnhsbmFBZWWhFQVAd1B4Zm1pOVl4e
VFDdmxYcUk1K2lhWGISSFM5V3p0Szg5ZVQrcW
45SXVyVWo5SnhTQklYMW1CUERnPT0ifQ==

Por fim, selecionamos a opção 5 - *Verificar assinatura de mensagem de A para B* e informamos a mensagem assinada recebida e a chave pública de A (remetente). Em resposta, obtivemos a confirmação de integridade e autenticidade da mensagem *Esta é uma mensagem de teste!*.

Realizamos as operações em máquina com as especificações descritas na Tabela I.

Tabela I
ESPECIFICAÇÕES DA MÁQUINA UTILIZADA PARA O EXPERIMENTO

Componente	Descrição
Frequência máxima de clock	2,1 GHz
Processador	12th Gen Intel(R) Core(TM) i7-1260P (Intel64 Family 6 Model 154 Stepping 3 GenuineIntel)
Número de núcleos	12
Número de processadores lógicos	16
Memória física total	33,94 GB
Sistema operacional	Microsoft Windows 11 Home Single Language x64-based PC (versão 10.0.26200)

A Tabela II indica o tempo médio de CPU consumido pelas principais operações executadas pelo programa.

Tabela II
TEMPO MÉDIO DE CPU

Operações	Iterações	Tempo médio de CPU
Geração de primo	100	3584,84 ms
Codificação OAEP	10000	0,00234 ms
Decodificação OAEP	10000	0,04531 ms
Cifragem RSA (mensagem)	10000	0,16406 ms
Cifragem RSA (assinatura)	100	33,125 ms
Decifragem RSA (mensagem)	100	21,71875 ms
Decifragem RSA (assinatura)	10000	0,19375 ms
Codificação PSS	10000	0,0375 ms
Decodificação PSS	10000	0,03906 ms

IV. CONCLUSÕES

O projeto desenvolvido demonstrou, em ambiente computacional convencional, a viabilidade técnica de implementar o sistema RSA (complementado pelos esquemas OAEP e PSS) como ferramenta de cifragem e de assinatura digital de mensagens.

Os testes evidenciaram que, enquanto as operações relativas aos esquemas OAEP e PSS são computacionalmente pouco custosas (ordem de 10^{-2} ms), o custo computacional de geração dos números primos que compõem as chaves do sistema RSA é significativo (ordem de 10^3 ms).

De outro, os testes mostraram que as operações de exponenciação modular que estão na base do sistema RSA têm custo computacional variável de acordo com a magnitude dos expoentes considerados.

No caso de teste, por exemplo, quando realizamos operações com o expoente e (ordem de 10^4), observamos tempo médio de CPU da ordem de 10^{-1} ms. Já quando operamos com o expoente d (ordem de 10^{230}), verificamos incremento no tempo médio de CPU da operação (ordem de 10^1 ms).

Adicionalmente, registramos a limitação do sistema RSA quanto ao tamanho da mensagem a ser cifrada, que deve ser menor que o valor de n . No caso de teste, em que n era da ordem de 2^{2048} , o teto de tamanho da mensagem era de 256 bytes ou 256 caracteres ASCII (sem considerar que os esquemas OAEP e PSS consomem muitos desses bytes). Esse fato evidencia as limitações em utilizar o sistema RSA como mecanismo de cifragem de mensagens extensas.

REFERÊNCIAS

- [1] Contribuidores da Wikipédia, *RSA cryptosystem*, Wikipedia, The Free Encyclopedia [Online]. 31 mar. 2026, 17h32min. UTC. Disponível: https://en.wikipedia.org/w/index.php?title=RSA_cryptosystem&oldid=1346408321.
- [2] B. S. Kaliski (Ed.), *PKCS #1: RSA Cryptography Specifications Version 2.2*, RFC 8017, Internet Engineering Task Force, 2016. [Online]. Disponível: <https://doi.org/10.17487/RFC8017>.
- [3] Contribuidores da Wikipédia, *Euler's totient function*, Wikipedia, The Free Encyclopedia [Online]. 28 mar. 2026, 13h33min. UTC. Disponível: https://en.wikipedia.org/w/index.php?title=Euler%27s_totient_function&oldid=1345837814.

APÊNDICE CÓDIGO DO PROGRAMA

A íntegra do código do programa desenvolvido pode ser consultada abaixo ou no repositório do projeto no GitHub: <https://github.com/andmilhomem/rsa>.

```

1
2 import secrets
3 import math
4 import base64
5 import hashlib
6 import json
7 import random
8 import time # Para testes
9
10 # Parâmetros:
11 TAM_HASH_BYTES = 32 # SHA-256
12 LABEL_HASH = b"" # Usada para gerar hash de
    ↳ início do pacote OAEP
13 TAM_P_Q_BITS = 1024 # O teto para n (e,
    ↳ portanto, para m) o dobro desse valor
14 E_CANDIDATO = 65537
15 TESTE = False # Alterar para ativar testes
16 ITERACOES_TESTE_PRIMO = 100
17 ITERACOES_TESTE_E = 10000
18 ITERACOES_TESTE_D = 100
19 ITERACOES_TESTE_ENC_DEC = 10000
20
21 # Funções auxiliares:
22 def int_para_base64(i):
23     return base64.b64encode(i.to_bytes((i.bit_
    ↳ _length() + 7) // 8, "big"))
24
25 def base64_para_int(b):
26     return int.from_bytes(base64.b64decode(b)
    ↳ , "big")
27
28 def xor_bytes(a, b):
29     return bytes(x ^ y for x, y in zip(a, b))
30
31 def aplica_miller_rebin(n):
32     k=40
33     # casos básicos
34     if n < 2:
35         return False
36     if n in (2, 3):
37         return True
38     if n % 2 == 0:
39         return False
40
41     # escreve n-1 = d * 2^r
42     d = n - 1
43     r = 0
44     while d % 2 == 0:
45         d //= 2
46         r += 1
47
48     # testes
49     for _ in range(k):
50         a = random.randrange(2, n - 2)
51         x = pow(a, d, n)
52
53         if x == 1 or x == n - 1:
54             continue
55
56         for _ in range(r - 1):
57             x = pow(x, 2, n)
58             if x == n - 1:
59                 break
60         else:

```



```

61         return False
62
63     return True
64
65 def aplica_mgf1(semente, tam_mascara):
66     resultado = b""
67     for cont in range(math.ceil(tam_mascara /
68     ↪ TAM_HASH_BYTES)):
69         cont_bytes = cont.to_bytes(4, "big")
70         resultado += hashlib.sha3_256(semente
71         ↪ + cont_bytes).digest()
72     return resultado[:tam_mascara]
73
74 def gera_primo(qtde_bits):
75     min_val = 1 << (qtde_bits - 1)
76     max_val = (1 << qtde_bits) - 1
77
78     e_primo = False
79     while not e_primo:
80         candidato = secrets.randbelow(max_val
81         ↪ - min_val + 1) + min_val # gera
82         ↪ candidato no intervalo desejado
83         candidato = candidato | 1 # transforma
84         ↪ em mpar, se necessario
85         e_primo =
86         ↪ aplica_miller_rebin(candidato) #
87         ↪ checa primalidade
88
89     return candidato
90
91 def gera_e(phi_n):
92     e = E_CANDIDATO
93     while not math.gcd(e, phi_n) == 1:
94         e += 2
95     return e
96
97 def formata_chave(n, e_ou_d, tipo):
98     # Compõe JSON
99     chave_formatada = ""
100     if (tipo == "publ"):
101         chave_formatada = json.dumps({
102             "n": int_para_base64(n).decode('u'
103             ↪ tf-8'),
104             "e": int_para_base64(e_ou_d).deco'
105             ↪ de('utf-8')
106         })
107     elif (tipo == "priv"):
108         chave_formatada = json.dumps({
109             "n": int_para_base64(n).decode('u'
110             ↪ tf-8'),
111             "d": int_para_base64(e_ou_d).deco'
112             ↪ de('utf-8')
113         })
114
115     # Converte JSON para base64
116     chave_formatada_base64 = base64.b64encode(
117     ↪ (chave_formatada.encode("utf-8")))
118
119     return
120     ↪ chave_formatada_base64.decode("utf-8")
121
122 def desformata_chave(chave):
123     # Decompõe JSON
124     chave_json = json.loads(base64.b64decode(
125     ↪ chave).decode("utf-8"))
126     n_base64 = chave_json.get("n")
127     d_base64 = chave_json.get("d")
128     e_base64 = chave_json.get("e")
129
130     if d_base64 != None:
131         return base64_para_int(n_base64),
132         ↪ base64_para_int(d_base64)
133     else:
134         return base64_para_int(n_base64),
135         ↪ base64_para_int(e_base64)
136
137 # Funções principais:
138 def gera_chaves():
139     if TESTE == True:
140         inicio = time.process_time_ns()
141         for i in range(ITERACOES_TESTE_PRIMO):
142             p = gera_primo(TAM_P_Q_BITS)
143             fim = time.process_time_ns()
144             print(f"\nTempo de
145             ↪ {ITERACOES_TESTE_PRIMO} geraes de
146             ↪ primos ({fim-inicio} ns):")
147
148     p = gera_primo(TAM_P_Q_BITS)
149     q = gera_primo(TAM_P_Q_BITS)
150     n = p*q
151     phi_n = (p-1)*(q-1)
152     e = gera_e(phi_n)
153     d = pow(e, -1, phi_n) # pow(a,b,c) = a^b
154     ↪ mod c, ou seja, d = e^(-1) mod phi_n
155     ↪ o inverso modular de e
156     return n, e, d
157
158 def cifra_decifra(base, expoente, n):
159     return pow(base, expoente, n)
160
161 def empacota_oaep(m, n):
162     # Calcula tamanho mximo da mensagem a
163     ↪ partir de 'n'
164     tam_n_em_bytes = (n.bit_length() + 7) // 8
165     tam_max_m_em_bytes = tam_n_em_bytes - 2 -
166     ↪ (2 * TAM_HASH_BYTES) # -1 (0x00 na EM
167     ↪ porque m < n), -1 (0x01 no DB como
168     ↪ delimitador), -TAM_HASH_BYTES (hash do
169     ↪ DB), -TAM_HASH_BYTES (masked_seed de
170     ↪ EM)
171
172     # Checa tamanho de 'm'
173     tam_m = len(m)
174     if tam_m > tam_max_m_em_bytes:
175         return -1, f"\nO tamanho da mensagem
176         ↪ ({tam_m} bytes) excedeu o limite
177         ↪ mximo ({tam_max_m_em_bytes}
178         ↪ bytes)!"
179
180     # Gera padding de zeros
181     tam_padding = tam_max_m_em_bytes - tam_m
182     padding = b"\x00" * tam_padding
183
184     # Gera hash do pacote
185     h = hashlib.sha3_256(LABEL_HASH).digest()
186
187     # Compõe pacote (gera data block)
188     pacote = h + padding + b"\x01" + m
189
190     # Mascara pacote (gera masked data block)
191     semente =
192     ↪ secrets.token_bytes(TAM_HASH_BYTES)
193     mascara_pacote = aplica_mgf1(semente,
194     ↪ len(pacote))
195     pacote_mascarado = xor_bytes(pacote,
196     ↪ mascara_pacote)
197
198     # Mascara semente (gera masked seed)

```

```

167 mascara_semente =
168     ↪ aplica_mgfl(pacote_mascarado,
169     ↪ TAM_HASH_BYTES)
170 semente_mascarada = xor_bytes(semente,
171     ↪ mascara_semente)
172
173 # Compe pacote final (gera encoded
174     ↪ message) e converte para int
175 pacote_final = int.from_bytes(b"\x00" +
176     ↪ semente_mascarada +
177     ↪ pacote_mascarado, "big")
178 return pacote_final, ""
179
180 def desempacota_oaep(pacote):
181     # Decompõe pacote
182     if pacote[0] != 0x00:
183         return -1, "início diferente de 0x00"
184     semente_mascarada = pacote[1: 1 +
185         ↪ TAM_HASH_BYTES]
186     pacote_mascarado = pacote[1 +
187         ↪ TAM_HASH_BYTES:]
188
189 # Desmascara semente
190 mascara_semente =
191     ↪ aplica_mgfl(pacote_mascarado,
192     ↪ TAM_HASH_BYTES)
193 semente_desmascarada =
194     ↪ xor_bytes(semente_mascarada,
195     ↪ mascara_semente)
196
197 # Desmascara pacote
198 mascara_pacote =
199     ↪ aplica_mgfl(semente_desmascarada,
200     ↪ len(pacote_mascarado))
201 pacote_desmascarado =
202     ↪ xor_bytes(pacote_mascarado,
203     ↪ mascara_pacote)
204
205 # Desempacota mensagem
206 h = pacote_desmascarado[0:TAM_HASH_BYTES]
207 resto =
208     ↪ pacote_desmascarado[TAM_HASH_BYTES:]
209 if h !=
210     ↪ hashlib.sha3_256(LABEL_HASH).digest():
211     ↪ # Checa hash
212     return -1, "hash inválido"
213 indice_separador = resto.find(b'\x01') #
214     ↪ Checa separador
215 if indice_separador == -1:
216     return -1, "separador não localizado"
217 padding = resto[:indice_separador]
218 m_desempacotada = resto[indice_separador +
219     ↪ 1:]
220 for b in padding: # Checa padding
221     if b != 0x00:
222         return -1, "padding inválido"
223 return 0, m_desempacotada.decode("utf-8")
224
225 def empacota_pss(m, n):
226     # Calcula tamanho mximo da assinatura
227     ↪ (encoded message) a partir de 'n'
228     tam_max_a_bits = n.bit_length() - 1
229     tam_max_a_bytes = (tam_max_a_bits + 7) //
230     ↪ 8
231
232 # Checa se os bits sobressalentes esto
233     ↪ zerados
234 bits_sobressalentes = (8 *
235     ↪ tam_max_a_bytes) - tam_max_a_bits
236 if bits_sobressalentes > 0:
237     mascara = 0xFF << (8 -
238         ↪ bits_sobressalentes) & 0xFF
239     if (pacote[0] & mascara) != 0:
240         return -1, "bits sobressalentes no
241         ↪ zerados"
242
243 # Decompõe pacote
244 if pacote[-1] != 0xbc:

```



```

262         return -1, "fim diferente de 0xbc"
263     pos_h_final = len(pacote) - TAM_HASH_BYTES
264     ↪ - 1
265     pacote_mascarado = pacote[:pos_h_final]
266     h_final_recebido = pacote[pos_h_final:
267     ↪ pos_h_final + TAM_HASH_BYTES]
268
269     # Desmascara pacote
270     h = hashlib.sha3_256(m).digest()
271     mascara_pacote =
272     ↪ aplica_mgf1(h_final_recebido, len
273     ↪ (pacote) - TAM_HASH_BYTES - 1)
274     pacote_desmascarado =
275     ↪ xor_bytes(pacote_mascarado,
276     ↪ mascara_pacote)
277
278     # Zera bits sobressalentes
279     if bits_sobressalentes > 0:
280         mascara = (0xFF >>
281         ↪ bits_sobressalentes)
282         ba = bytearray(pacote_desmascarado)
283         ba[0] &= mascara
284         pacote_desmascarado_ajustado =
285         ↪ bytes(ba)
286     else:
287         pacote_desmascarado_ajustado =
288         ↪ pacote_mascarado
289
290     # Checa se padding está zerado
291     tam_padding_pacote_bytes = tam_max_a_bytes
292     ↪ - ( 2 * TAM_HASH_BYTES) - 2 # em
293     ↪ menos (H + 0xbc) menos (0x01 + salt)
294     padding = pacote_desmascarado_ajustado[:t]
295     ↪ am_padding_pacote_bytes]
296     for b in padding: # Checa padding
297         if b != 0x00:
298             return -1, "padding inválido"
299
300     # Checa se separador está correto
301     if pacote_desmascarado_ajustado[tam_paddi]
302     ↪ ng_pacote_bytes] != 1:
303         return -1, "separador não localizado"
304
305     # Calcula h_final
306     salt = pacote_desmascarado_ajustado[tam_p]
307     ↪ adding_pacote_bytes+1:]
308     h = hashlib.sha3_256(m).digest() #mHash
309     m_linha = b"\x00" * 8 + h + salt
310     h_final =
311     ↪ hashlib.sha3_256(m_linha).digest()
312
313     # Verifica assinatura
314     return 0, h_final_recebido == h_final
315
316 def assina_mensagem(m, n, d):
317     # Gera e empacota assinatura
318     a = empacota_pss(m, n)
319     if a == -1: return a, "Erro ao assinar!"
320     if TESTE == True:
321         inicio = time.process_time_ns()
322         for i in
323         ↪ range(ITERACOES_TESTE_ENC_DEC):
324             empacota_pss(m, n)
325         fim = time.process_time_ns()
326         print(f"\nTempo de
327         ↪ {ITERACOES_TESTE_ENC_DEC}
328         ↪ codificaes PSS ({fim-inicio}
329         ↪ ns):")
330
331     # Cifra assinatura
332
333     a_cifrada = cifra_decifra(a, d, n)
334     if TESTE == True:
335         inicio = time.process_time_ns()
336         for i in range(ITERACOES_TESTE_D):
337             cifra_decifra(a, d, n)
338         fim = time.process_time_ns()
339         print(f"\nTempo de
340         ↪ {ITERACOES_TESTE_D} cifragens RSA
341         ↪ ({fim-inicio} ns):")
342     a_cifrada_base64 =
343     ↪ int_para_base64(a_cifrada)
344
345     # Compõe mensagem e assinatura
346     m_assinada = json.dumps({
347         "mensagem": m.decode("utf-8"),
348         "assinatura":
349         ↪ a_cifrada_base64.decode('utf-8')
350     })
351
352     # Converte mensagem para base64
353     m_assinada_base64 = base64.b64encode(m_as)
354     ↪ sinada.encode("utf-8")
355
356     return 0,
357     ↪ m_assinada_base64.decode("utf-8")
358
359 while True:
360     print("\n===== OPES
361     ↪ =====\n")
362     print("1 - Gerar chaves pblicas e privadas
363     ↪ para A e B")
364     print("2 - Cifrar mensagem de A para B")
365     print("3 - Decifrar mensagem de A para B")
366     print("4 - Assinar mensagem de A para B")
367     print("5 - Verificar assinatura de
368     ↪ mensagem de A para B")
369     print("\n=====
370     ↪ =====\n")
371
372     try:
373         match input("\n> "):
374             case "1": # Gera chaves
375                 n_A, e_A, d_A = gera_chaves()
376                 n_B, e_B, d_B = gera_chaves()
377
378             try:
379                 chave_pblica_A =
380                 ↪ formata_chave(n_A,
381                 ↪ e_A, "publ")
382                 chave_privada_A =
383                 ↪ formata_chave(n_A,
384                 ↪ d_A, "priv")
385                 chave_pblica_B =
386                 ↪ formata_chave(n_B,
387                 ↪ e_B, "publ")
388                 chave_privada_B =
389                 ↪ formata_chave(n_B,
390                 ↪ d_B, "priv")
391
392                 print("\n=====
393                 ↪ CHAVE PBLICA DE A
394                 ↪ =====\n")
395                 print(chave_pública_A)
396                 print("\n=====
397                 ↪ CHAVE PRIVADA DE A
398                 ↪ =====\n")
399                 print(chave_privada_A)

```

```

359         print("\n=====
        ↪ CHAVE PÚBLICA DE B
        ↪ =====\n")
360     print(chave_pública_B)
361     print("\n=====
        ↪ CHAVE PRIVADA DE B
        ↪ =====\n")
362     print(chave_privada_B)
363     print("\n=====]
        ↪ =====]
        ↪ =====\n")
364     except:
365         print("\nErro ao gerar
        ↪ chaves!")
366
367     case "2": # Cifra mensagem
368         # Obtém 'chave'
369         print("\nInforme a chave
        ↪ pública de B:")
370         try:
371             n_B, e_B = desformata_cha_
        ↪ ve(input("> "))
372         except:
373             print("\nErro ao processar
        ↪ chave!")
374             continue
375
376         # Obtém 'm'
377         print(f"\nInforme a
        ↪ mensagem:")
378         m = input(">
        ↪ ").encode("utf-8")
379
380         # Empacota 'm'
381         pacote, informacao =
        ↪ empacota_oaep(m, n_B)
382         if TESTE == True:
383             inicio =
        ↪ time.process_time_ns()
384             for i in range(ITERACOES_
        ↪ TESTE_ENC_DEC):
385                 empacota_oaep(m, n_B)
386             fim =
        ↪ time.process_time_ns()
387             print(f"\nTempo de {ITERA_
        ↪ COES_TESTE_ENC_DEC}
        ↪ codificaes OAEP
        ↪ ({fim-inicio} ns):")
388         if pacote == -1:
389             print(informacao)
390             continue
391
392         # Cifra pacote (RSA)
393         c = cifra_decifra(pacote, e_B,
        ↪ n_B)
394         print(f"\nMensagem cifrada com
        ↪ sucesso:\n{int_para_base6_
        ↪ 4(c).decode('utf-8')}")
395         if TESTE == True:
396             inicio =
        ↪ time.process_time_ns()
397             for i in range(ITERACOES_
        ↪ TESTE_E):
398                 cifra_decifra(pacote,
        ↪ e_B, n_B)
399             fim =
        ↪ time.process_time_ns()
400
401         print(f"\nTempo de
        ↪ {ITERACOES_TESTE_E}
        ↪ cifraes RSA
        ↪ ({fim-inicio} ns):")
402
403     case "3": # Decifra mensagem
404         # Obtém 'chave'
405         print("\nInforme a chave
        ↪ privada de B:")
406         try:
407             n_B, d_B = desformata_cha_
        ↪ ve(input("> "))
408         except:
409             print("\nErro ao processar
        ↪ chave (formato
        ↪ invlido)!")
410             continue
411
412         # Obtém mensagem cifrada
413         print(f"\nInforme a mensagem
        ↪ cifrada (em base64):")
414         try:
415             c = base64_para_int(input_
        ↪ "> ")
416         except:
417             print("\nErro ao processar
        ↪ mensagem (formato
        ↪ invlido)!")
418             continue
419
420         # Decifra mensagem
421         pacote_decifrado_int =
        ↪ cifra_decifra(c, d_B, n_B)
422         if TESTE == True:
423             inicio =
        ↪ time.process_time_ns()
424             for i in range(ITERACOES_
        ↪ TESTE_D):
425                 cifra_decifra(c, d_B,
        ↪ n_B)
426             fim =
        ↪ time.process_time_ns()
427             print(f"\nTempo de
        ↪ {ITERACOES_TESTE_D}
        ↪ decifragens RSA
        ↪ ({fim-inicio} ns):")
428             tam_n_em_bytes =
        ↪ (n_B.bit_length() + 7) //
        ↪ 8
429             pacote_decifrado_bytes =
        ↪ pacote_decifrado_int.to_b_
        ↪ ytes(tam_n_em_bytes, "big")
430             resultado, informacao =
        ↪ desempacota_oaep(pacote_d_
        ↪ ecifrado_bytes)
431         if TESTE == True:
432             inicio =
        ↪ time.process_time_ns()
433             for i in range(ITERACOES_
        ↪ TESTE_ENC_DEC):
434                 desempacota_oaep(paco_
        ↪ te_decifrado_byte_
        ↪ s)
435             fim =
        ↪ time.process_time_ns()
436             print(f"\nTempo de {ITERA_
        ↪ COES_TESTE_ENC_DEC}
        ↪ decodificaes OAEP
        ↪ ({fim-inicio} ns):")

```

```

437         if resultado == -1:
438             print(f"\nMensagem
439                 ↳ corrompida
440                 ↳ ({informacao})!")
441             continue
442
443         print(f"\nMensagem decifrada
444             ↳ com
445             ↳ sucesso:\n{informacao}")
446
447     case "4": # Assina mensagem
448         # Obtém 'chave'
449         print("\nInforme a chave
450             ↳ privada de A:")
451         try:
452             n_A, d_A = desformata_cha
453             ↳ ve(input("> "))
454         except:
455             print("\nErro ao processar
456                 ↳ chave!")
457             continue
458
459         # Obtém 'm'
460         print(f"\nInforme a
461             ↳ mensagem:")
462         m = input("> ")
463         ↳ ".encode("utf-8")
464
465         # Assina mensagem
466         resultado, informacao =
467         ↳ assina_mensagem(m, n_A,
468         ↳ d_A)
469         if resultado == -1:
470             ↳ print(informacao)
471         else: print(f"\nMensagem
472             ↳ assinada com
473             ↳ sucesso:\n{informacao}")
474
475     case "5": # Verifica assinatura
476         # Obtém 'chave'
477         print("\nInforme a chave
478             ↳ pblica de A:")
479         try:
480             n_A, e_A = desformata_cha
481             ↳ ve(input("> "))
482         except:
483             print("\nErro ao processar
484                 ↳ chave!")
485             continue
486
487         try:
488             # Obtm e decompoe a
489             ↳ mensagem
490             print(f"\nInforme a
491                 ↳ mensagem assinada (em
492                 ↳ base 64):")
493             m_recebida = base64.b64de
494             ↳ code(input("> ")
495             ↳ ").decode("utf-8")
496             m_recebida_json =
497             ↳ json.loads(m_recebida)
498             m_recebida_bytes =
499             ↳ m_recebida_json["mens
500             ↳ agem"].encode("utf-8")
501             a_recebida_base64 =
502             ↳ m_recebida_json["assi
503             ↳ natura"]
504
505             # Decifra assinatura
506             ↳ recebida
507
508             a_recebida_int =
509             ↳ base64_para_int(a_rec
510             ↳ ebida_base64)
511             a_decifrada_int = cifra_d
512             ↳ ecifra(a_recebida_int,
513             ↳ e_A, n_A)
514             if TESTE == True:
515                 inicio = time.process
516                 ↳ _time_ns()
517                 for i in range(ITERAC
518                 ↳ OES_TESTE_E):
519                     cifra_decifra(a_r
520                     ↳ ecebida_int,
521                     ↳ e_A, n_A)
522                 fim = time.process_ti
523                 ↳ me_ns()
524                 print(f"\nTempo de {I
525                 ↳ TERACOEES_TESTE_E}
526                 ↳ decifragens RSA
527                 ↳ ({fim-inicio}
528                 ↳ ns):")
529             except:
530                 print("\nErro ao processar
531                     ↳ mensagem assinada
532                     ↳ (formato invlido)!")
533                 continue
534
535             # Desempacota assinatura
536             ↳ recebida
537             tam_n_em_bytes =
538             ↳ (n_A.bit_length() + 7) //
539             ↳ 8
540             a_decifrada_bytes =
541             ↳ a_decifrada_int.to_bytes(
542             ↳ tam_n_em_bytes, "big")
543             resultado, informacao =
544             ↳ desempacota_pss(a_decifra
545             ↳ da_bytes,
546             ↳ m_recebida_bytes, n_A)
547             if TESTE == True:
548                 inicio =
549                 ↳ time.process_time_ns()
550                 for i in range(ITERACOEES_
551                 ↳ TESTE_ENC_DEC):
552                     desempacota_pss(a_dec
553                     ↳ ifrada_bytes,
554                     ↳ m_recebida_bytes,
555                     ↳ n_A)
556                 fim =
557                 ↳ time.process_time_ns()
558                 print(f"\nTempo de {ITERA
559                 ↳ COES_TESTE_ENC_DEC}
560                 ↳ decodificaes PSS
561                 ↳ ({fim-inicio} ns):")
562
563             if resultado == -1:
564                 print(f"\nAssinatura
565                     ↳ corrompida
566                     ↳ ({informacao})!")
567                 continue
568             if informacao == True:
569                 print("\nIntegridade
570                     ↳ confirmada da seguinte
571                     ↳ mensagem:")
572                 print(m_recebida_bytes.de
573                     ↳ code("utf-8"))
574             else:
575                 print("\nAssinatura
576                     ↳ invlida!")

```

```
511         case _:
512             print("\n Opo invlida. Tente
513                   ↳ novamente.\n")
514 except Exception as e:
515     print("Um erro inesperado ocorreu: ",
516           ↳ e)
```